

ClearClause: Final Project Report

Ethan Chelly GenAI & Machine Learning Bootcamp, Developer Institute July 2026

1. The problem

Most people sign leases, accept terms of service and buy insurance without really reading them. The language is dense, the documents are long, and nobody has time. The risky part is that the answers to practical questions (“can my landlord enter my apartment whenever he wants?”) are actually written in these documents, we just never look them up.

Large language models seem like an obvious fix, but asking a chatbot a legal question is dangerous: the model answers with confidence whether it knows or not. For legal content, a wrong answer that sounds right is worse than no answer.

ClearClause is my attempt to solve this properly. It is a Retrieval-Augmented Generation (RAG) application: you ask a question in plain English, it finds the relevant clauses in the actual document, and it answers in simple language while citing the exact source passage and page. If the document does not contain the answer, it says so instead of guessing.

2. What I built

The MVP covers the full RAG loop:

- PDF ingestion with sentence-aware chunking
- Semantic search over chunks (sentence-transformers embeddings + a FAISS index)
- Answer generation grounded in the retrieved passages, with source and page citations
- A Gradio chat interface with conversation history

On top of that I added the bonus features from my project scope:

- A side-by-side “RAG vs LLM-only” comparison tab
- An evaluation dashboard showing accuracy metrics against a ground-truth Q&A set
- Multi-document support with a document selector (lease, terms of service, insurance)
- A LoRA fine-tuning experiment on the legal Q&A data

Everything runs from one script (`./run.sh`) and the repository is deploy-ready for Hugging Face Spaces.

3. Data

I only used public-domain material, so there are no confidentiality issues:

- A model lease adapted from HUD form 90105a (US government work)
- A generic website terms of service assembled from public-domain US government site policies
- A flood insurance summary adapted from the FEMA National Flood Insurance Program Summary of Coverage

Three documents, three different “types” of legal text, 73 chunks in total. For evaluation I wrote a ground-truth set of 20 question/answer pairs (10 on the lease, 5 on the terms of service, 5 on the insurance document), each with the expected source document and page.

4. How the pipeline works

Ingestion. Each PDF is read page by page with pypdf, then split into chunks of at most 500 characters. My first version cut the text at fixed character positions, but that often cut sentences in half, which hurts both the embeddings and the readability of the citations shown to the user. The final version packs whole sentences into each chunk and carries the last sentences over into the next chunk as overlap. Every chunk keeps its source filename and page number, which is what makes exact citations possible later.

Embeddings and index. Each chunk is encoded with all-MiniLM-L6-v2 (small, fast, runs on CPU). The vectors are L2-normalized and stored in a FAISS IndexFlatIP index, so the inner product search is equivalent to cosine similarity, which is the metric this model was trained for. At this scale (73 vectors) an exact brute-force index is the right choice: simple and with zero approximation error.

Retrieval. A question is embedded the same way and the top 3 most similar chunks are retrieved. I added a minimum similarity threshold (0.25 cosine) because FAISS always returns the nearest neighbors, even for a question that has nothing to do with the corpus. With the threshold, an off-topic question gets an honest “I could not find anything relevant” instead of forcing the LLM to improvise on unrelated clauses. The search also supports filtering by document, which powers the document selector in the interface.

Generation. The retrieved chunks are assembled into a prompt together with the question and a system prompt that forbids using outside knowledge and requires citing source and page. Generation goes through the Hugging Face Inference API (Qwen2.5-7B-Instruct, temperature 0.2). When the API is not available, either because there is no token or because the free credits are used up, the pipeline automatically falls back to a local model (Qwen2.5-1.5B-Instruct through transformers). This means the demo cannot break because of an external quota.

Interface. The Gradio app has three tabs: the chat (with history, so follow-up questions work), the RAG vs LLM-only comparison, and the evaluation dashboard. The embedding model and the index are loaded once at startup instead of once per question, which took the answer latency from several seconds of overhead down to almost none.

5. Evaluation

The point of the evaluation is to prove that the RAG pipeline adds value over the same model used without retrieval. For each of the 20 ground-truth questions I generate two answers: one with the full pipeline and one where the same LLM answers from its own knowledge. I measure:

- ROUGE-1 F1 between the generated answer and the expected answer
- Whether the retrieved chunks come from the correct document (document hit rate)
- Whether they include the exact expected page (page hit rate)

Results:

METRIC	RAG	LLM-ONLY BASELINE
Average ROUGE-1 F1	0.448	0.202
Document hit rate	100%	n/a
Exact page hit rate	85%	n/a

The RAG pipeline more than doubles the answer quality of the same model, and retrieval finds the right document for every single question. The three questions that miss the exact page all involve clauses that span a page boundary: the right chunk is retrieved, it just carries the number of the page where it starts.

Two honest caveats. First, ROUGE measures n-gram overlap, not correctness: on two of the twenty questions the baseline actually scores slightly higher, because the expected answer is a short “no” and ROUGE penalizes the RAG answer for adding its cited explanation. Second, these numbers were produced with the local 1.5B fallback model (my API credits were

exhausted at that point), which if anything makes the result stronger: even a small model doubles its quality when it is grounded in the right passages.

6. Bonus: LoRA fine-tuning

I also ran the optional fine-tuning experiment from my scope. Using PEFT, I fine-tuned Qwen2.5-0.5B-Instruct on the Q&A pairs with LoRA adapters on the attention projections (rank 8). Only 540,672 parameters are trained, which is 0.11% of the 494M parameter model, and the training runs in seconds on my laptop.

The result on the two held-out questions is the most instructive part of the project. The fine-tuned model clearly picked up the *style* of the expected answers: short, direct, definite. But it got the facts wrong, for example answering “30 days” where the document says 60. With only 18 training pairs this is expected, and it illustrates the design of the whole project: fine-tuning shapes how a model answers, retrieval grounds what it answers. The two techniques complement each other, they do not compete.

7. Problems I ran into

A few real-world issues shaped the final architecture:

- The generation model I had planned (Mistral-7B-Instruct-v0.3) was removed from the Inference API chat service mid-project. I probed the available models and switched to Qwen2.5-7B-Instruct.
- My free Inference API credits ran out during evaluation. Instead of paying, I built the automatic local fallback, which turned an outage into a feature: the app now works with zero external dependencies.
- pip tried to compile FAISS from source on an Intel Mac because recent faiss-cpu releases stopped shipping binaries for that platform. I fixed it with an environment marker in requirements.txt that pins an older version only on Intel macOS.
- Hugging Face made Gradio Spaces hosting a paid (PRO) feature in 2026. The repository is fully deploy-ready (entry point, Space config, encrypted secret handling are all in place and documented), so deployment is one subscription away, and the demo runs locally in the meantime.

8. Limitations and future work

- The corpus is three documents. The architecture handles more (the ingestion just scans a folder), but a larger and more diverse corpus would make the retrieval evaluation more meaningful.
- ROUGE is a weak proxy for answer correctness. A next step would be an LLM-as-judge evaluation or human grading.
- Chunks that span page boundaries are attributed to their starting page. Storing a page range per chunk would fix the three “page miss” cases.
- Scanned PDFs are skipped (no OCR). Adding an OCR step would widen the range of usable documents.
- Everything is in English. Supporting French documents mainly means switching to a multilingual embedding model.

9. Conclusion

ClearClause does what the project scope promised: it makes legal documents queryable in plain language, it always shows its sources, and it comes with measurements instead of impressions. The evaluation shows retrieval is the part that makes the answers trustworthy, and the LoRA experiment shows why fine-tuning alone would not have been enough. The part I am most satisfied with is that every claim the assistant makes can be checked against the exact clause it came from, which is the whole point when the subject is a contract you are about to sign.